Prog 2 – Klausur SS 2025

Aufgabe für Aufgabe, plus alle MC-Fragen

Was hier drin steht

Teil 1: Die Klausur SS 2025, Aufgabe für Aufgabe – aus dem Gedankenprotokoll, ergänzt um die Vulkanaufgabe aus Übungsblatt 8, die der Protokollschreiber ausdrücklich empfiehlt.

Teil 2: Alle Multiple-Choice-Fragen, die in mehreren Klausuren vorkamen, mit Antworten und kurzer Begründung.

Teil 3: Die Tabellen zum Nachschlagen – Collections, Streams, funktionale Interfaces.

Wo Material aus Übungsblättern oder älteren Klausuren stammt, steht es dabei.

Teil 1 – Die Klausur SS 2025, Aufgabe für Aufgabe

Quellenlage – lies das kurz

Die aktuellste Klausur, für die Material existiert, ist SS 2025. Es gibt davon aber keine echte Klausur, sondern nur ein Gedankenprotokoll eines Studenten.

Belastbar dokumentiert sind: der komplette Aufbau, 25 Multiple-Choice-Fragen und sechs Code-Ausgabe-Aufgaben. Bei Aufgabe 3 hat der Protokollschreiber das UML-Diagramm aus dem Gedächtnis rekonstruiert und selbst dazugeschrieben, dass er sich nicht sicher ist. Aufgabe 4 konnte er gar nicht wiedergeben – er schreibt nur, es seien Stream-Aufgaben gewesen, und empfiehlt die Übungsaufgaben mit den Vulkandaten.

Deshalb ist Aufgabe 4 unten mit genau diesen Vulkandaten aus Übungsblatt 8 aufgefüllt. Das ist eine begründete Rekonstruktion, keine echte Klausuraufgabe – aber die beste Näherung, die es gibt.

Aufgabe	Inhalt	Dein Ziel
1	Multiple Choice, 25 Fragen zu Java und C	möglichst alle – siehe Teil 2
2	Offene Fragen und Code-Ausgaben	auswendig, schnelle Punkte
3	Java-Programmierung nach UML-Diagramm	zweimal üben, größter Block
4	Streams, zum Schluss eine C-Aufgabe	die Ketten sicher können

Was Esch selbst auf seiner Folie sagt

```
Teil 1: Wissensfragen zu Java und C, Multiple Choice
Teil 2: Java
ggf. Teil 3: C   (eine oder zwei EINFACHE C-Aufgaben)

Gewichtung:  Lambda+++   Generics+++   Collections+++
             C-Aggregattypen++   Pointer++
             Rekursion+  Innere Klassen+  C-Grundlagen+
```

Dazu wörtlich: Schauen Sie sich insbesondere die Übungen zu Collections, Generics und Lambda-Ausdrücken nochmal an. Sie sollten einfache Klassen anhand eines UML-Diagramms implementieren können und eine generische Klasse implementieren und nutzen können.

Aufgabe 1 – Multiple Choice

25 Fragen. Themen laut Protokoll: Rekursion (Ende, Speicherort), innere Klassen (Arten, Zugriff, lokale und anonyme Klassen), Generics (Eigenschaften, Objekterzeugung, Raw Types, Bounded Types), Lambda-Syntax, funktionale Interfaces, ArrayList und List, Terminal-Operationen, C-Eigenschaften, Linker, struct, Pointer-Adressen, Methodenreferenzen.

Wichtig aus dem Protokoll: Wird eine falsche Antwort angekreuzt oder sind nicht alle richtigen markiert, gibt es keinen Punkt – aber es wird nichts abgezogen. Also jedes Kästchen bearbeiten.

Alle Fragen mit Antworten stehen in Teil 2 dieses Hefts.

Teil 2 zweimal durchgehen: einmal ankreuzen, einmal nur Antworten lesen.

Durchgänge: ☐ ☐

Aufgabe 2 – Offene Fragen und Code-Ausgaben

Wörtlich aus dem Protokoll. Das sind die schnellsten Punkte der Klausur.

2.1 Was gibt folgender Code aus?

```
int result = Stream.of(1, 2, 3).reduce(1, (x, y) -> x * y);
```

```
System.out.println(result);
```

Ausgabe

6

reduce(1, …) startet mit dem Wert 1: 1·1 = 1, dann 1·2 = 2, dann 2·3 = 6. Der erste Parameter ist der Startwert, nicht das erste Element.

2.2 Welchen Wert hat x nach der Ausführung?

```
int a[5] = {10, 20, 30, 40, 50};
int *p = a;
int x = *(p + 2);
printf("%d\n", x);
```

Ausgabe

30

p zeigt auf das erste Element, p+2 springt zwei Elemente weiter. Achtung: *(p)+2 wäre 12.

2.3 Welche Ausgabe erzeugt folgender Code?

```
List<Integer> zahlen = Arrays.asList(1, 2, 3);
boolean result = zahlen.stream().anyMatch(x -> x > 2);
System.out.println(result);
```

Ausgabe

true

anyMatch liefert true, sobald mindestens ein Element passt – hier die 3.

2.4 Fragen in ganzen Sätzen beantworten

Musterantworten

a) Was ist ein funktionales Interface in Java? Ein Interface mit genau einer abstrakten Methode. Dadurch kann es durch einen Lambda-Ausdruck oder eine Methodenreferenz implementiert werden. Zusätzliche default- und static-Methoden sind erlaubt.

b) Was passiert, wenn eine Rekursion unendlich weiterläuft? Für jeden Aufruf wird ein neuer Stack-Frame angelegt. Da der Stack begrenzt ist, läuft er über und die JVM wirft einen StackOverflowError.

c) Wo speichert eine Rekursion ihre aktuellen Werte ab? Auf dem Stack. Jeder Aufruf hat einen eigenen Stack-Frame mit eigenen lokalen Variablen und Parametern.

d) Welcher Operator liefert in C die Adresse einer Variablen? Der Adress-Operator &.

e) Welcher Operator dereferenziert in C einen Pointer? Der Stern-Operator *.

f) Welches funktionale Interface nimmt keine Argumente und gibt ein Ergebnis zurück? Supplier<T> mit der Methode T get().

2.5 Welche Ausgabe erzeugt folgendes Programm?

```
public static int fac(int n) {
    if (n <= 1) { System.out.println("B"); return 1; }
    else return n * fac(n - 1);
}
// main: int result = fac(3); System.out.println(result);
```

Ausgabe

B

6

Das B wird nur im Basisfall gedruckt, also genau einmal bei fac(1). Danach wird zurückgerechnet: 1, dann 2·1 = 2, dann 3·2 = 6.

2.6 Welche Ausgaben erscheinen auf der Konsole?

```
public class Outer {
    public static int i = 0;
    public void testLocalClass() {
        class Inner {
            private int i = 1;
            public void print(int i) {
                System.out.println("i: " + i);
                System.out.println("i: " + this.i);
                System.out.println("i: " + Outer.i);
            }
        }
        new Inner().print(2);
    }
}
```

Ausgabe

i: 2
i: 1
i: 0

i allein ist der Parameter. this.i ist das Feld der inneren Klasse. Outer.i ist das statische Feld der äußeren Klasse.

Alle sechs Aufgaben zweimal durchgehen, Ausgaben aus dem Kopf.

Durchgänge: ☐ ☐

Aufgabe 3 – Java-Programmierung nach UML

Dein größter Block

Diese Aufgabe war 2024, 2024/25 und 2025 immer dieselbe Form: abstrakte Oberklasse, konkrete Unterklassen, eine Verwalterklasse mit einer Collection. Nur die Namen wechseln – SmartDevice, Weapon, Shape.

Zweimal schreiben. Wenn nur eine Sache sitzt, dann diese.

Aufgabenstellung SS 2025 (rekonstruiert)

Gegeben ist ein UML-Diagramm mit einer abstrakten Klasse Shape und der konkreten Subklasse Circle.

```
abstract Shape
  + xy    : Point2D
  + color : String
  + Shape(xy, color)
  + toString() : String

Circle extends Shape          Rectangle extends Shape
  - radius : int                + Rectangle(...)
  + toString() : String         + getString()

Point2D
  - x : int   - y : int
  + compareTo(Point2D) : int
  + getX() : int

Canvas
```

```
  + addShape(Shape s) : void
  + getShapeSorted() : List<Shape>
```

a) Klassen-Implementierung: Implementieren Sie die Klasse Circle in Java.

b) Point2D Comparable machen: Implementieren Sie eine Klasse Point2D mit den Attributen x und y.

c) Canvas-Klasse: Implementieren Sie eine Klasse Canvas, die intern eine TreeMap<Point2D, Shape> nutzt.

Was die Aufgabe dir verrät

Teilaufgabe c verlangt eine TreeMap mit Point2D als Schlüssel. Eine TreeMap braucht eine Ordnung auf ihren Schlüsseln – deshalb steht in b, dass Point2D Comparable werden soll. Die beiden Teilaufgaben hängen zusammen.

Merkregel für die Klausur: Sobald irgendwo TreeMap, TreeSet oder das Wort sortiert steht, ist Comparable die versteckte Pflichtaufgabe.

Die Reihenfolge beim Schreiben

- Shape: Felder, Konstruktor, Getter, abstrakte Methode, toString
- Circle: eigenes Feld, Konstruktor mit super(...) als erster Zeile, Getter, toString mit super.toString()
- Point2D: Felder, Konstruktor, Getter, compareTo (erst x, bei Gleichstand y), equals, hashCode, toString
- Canvas: TreeMap als Feld, addShape mit put, getShapeSorted gibt new ArrayList<>(map.values()) zurück

Bei compareTo mit zwei Kriterien: erst prüfen, ob sich die x unterscheiden – wenn ja entscheidet x, sonst y. Mit Integer.compare, nicht mit Subtraktion.

Verifizierte Zielausgabe

```
getShapeSorted() bei den Punkten (3,4), (1,9), (1,2):
  Rectangle (1,2) gruen 4x6
  Circle    (1,9) blau  r=2
  Circle    (3,4) rot   r=5

TreeSet der Punkte -> [(1,2), (1,9), (3,4)]     erst nach x, dann nach y
```

An zwei Tagen je einmal komplett schreiben.

Durchgänge: ☐ ☐

Ergänzende Übung: die Raumreservierung

Dieselbe Aufgabenform, aber größer und mit Vererbung über zwei Ebenen. Quelle: Übungsblatt 1, Zusatzaufgabe 1. Gut, weil das UML-Diagramm hier vollständig vorliegt und nicht rekonstruiert ist.

Das Diagramm in Textform

```
Uhrzeit
  - stunde : int      - minute : int
  + Uhrzeit(stunde, minute)      + toString() : String

Person
  - vorname : String  - nachname : String
  + Person(vorname, nachname)    + toString() : String

Mitarbeiter extends Person
  - email : String
  + Mitarbeiter(vorname, nachname, email)   + toString() : String
  + reserviere(raum, beginn, ende, bemerkung)

Reservierung
  - bemerkung : String   - beginn : Uhrzeit   - ende : Uhrzeit
  + Reservierung(beginn, ende)
```

```
  + setBemerkung(...)  + setMitarbeiter(...)  + setRaum(...)  + toString()

Raum
  - geb : int   - etage : int   - raum : int
  + Raum(geb, etage, raum)   + toString()   + addReservierung(reservierung)

Beziehungen:  Mitarbeiter 1 -- 0..* Reservierung
              Raum        1 -- 0..* Reservierung
```

Geforderte Ausgaben

```
new Uhrzeit(12, 54)                          ->  12:54 Uhr
new Mitarbeiter("Max","Mustermann","max@...")  ->  Max Mustermann (max@htwsaar.de)
new Raum(18, 0, 1)                           ->  Raum 18-0.1

System.out.println(r1) nach einer Reservierung ->
  Raum 18-0.1
  gebucht von Max Mustermann (max@htwsaar.de) von 12:30 Uhr bis 14:30 Uhr für VOOP
```

Worauf du achten musst

- reserviere() legt die Reservierung an, setzt Bemerkung, Mitarbeiter und Raum – und hängt sie beim Raum ein. Sonst fehlt sie in der Ausgabe.
- Die Uhrzeit braucht String.format mit %02d für die Minuten, sonst steht dort 12:5 statt 12:05.
- Raum.toString baut den Grundtext und hängt jede Reservierung in einer neuen Zeile an, am einfachsten mit StringBuilder.
- Doppelbelegungen und Sortierung musst du laut Aufgabe NICHT prüfen.

An drei verschiedenen Tagen je einmal.

Durchgänge: ☐ ☐ ☐

Aufgabe 4 – Streams

Warum genau diese Aufgaben

Der Protokollschreiber schreibt wörtlich: Bei Aufgabe 4 ging es um Stream-Aufgaben. Ich empfehle, die Übungsblätter durchzuarbeiten, besonders die Aufgaben mit den Vulkandaten oder den Schüler-Beispielen sind sehr hilfreich und repräsentativ für die Klausur.

Die Vulkanaufgabe ist Übungsblatt 8, Aufgabe 1 aus deinem Semester. Die fünfzehn Abfragen unten sind die Originalliste von dort.

Der Datensatz

```
record Eruption(int volcanoNumber, String volcanoName, int eruptionNumber,
                int startYear, int endYear, int dauer,
                String evidenceMethod, double latitude, double longitude) {}

// Zum Ueben reichen fuenf bis sechs Eintraege, die du selbst anlegst.
```

Die fünfzehn Abfragen (Originalliste aus Übungsblatt 8)

- Anzahl aller Eruptionen
- Durchschnittliche Dauer in Tagen
- Letzte bekannte Eruption für alle Vulkane
- Alle Eruptionen vor dem Jahr 1000
- Top 10 Jahre mit den meisten Eruptionen
- Kategorisierung der Ausbrüche nach Evidence Method
- Anzahl Eruptionen nördlich des Äquators
- Anzahl Eruptionen südlich des Äquators

- Anzahl Eruptionen je Dekade
- Anzahl der Eruptionen je Vulkan
- Eruptionen mit Start- und Enddatum im selben Jahr
- Verteilung der Eruptionsdauer: Klassen 0-1, 2-5, 6-10, 10+
- Längste bekannte Eruption
- Evidence Methods, sortiert nach Häufigkeit
- 10 Vulkane mit kürzesten Eruptionen

Verifizierte Ergebnisse auf einem Sechs-Eintrags-Datensatz

```
1  Anzahl               6
2  Durchschnittsdauer   32.17
3  Letzte je Vulkan     {Etna=2001, Merapi=2010, Taupo=232, Fuji=1707}
4  Vor Jahr 1000        [Taupo]
6  Evidence Methods     [Radiocarbon, Seismicity, Historical]
7  Noerdlich            3
8  Suedlich             3
9  Je Dekade            {2000=1, 1700=1, 230=1, 1990=2, 2010=1}
10 Je Vulkan            {Etna=2, Merapi=2, Taupo=1, Fuji=1}
11 Start=Ende           4
12 Dauerklassen         {2-5=2, 10+=3, 6-10=1}
13 Laengste             Merapi
14 Methods n. Haeufigk. [Historical=3, Seismicity=2, Radiocarbon=1]
15 3 kuerzeste          [Taupo, Etna, Merapi]
```

Die vier Muster, aus denen alle fünfzehn bestehen

```
zaehlen / rechnen:
  .count()   .mapToInt(X::feld).sum()   .average().orElse(0)

filtern und abbilden:
  .filter(p).map(X::feld).toList()

gruppieren:
  .collect(Collectors.groupingBy(X::feld, Collectors.counting()))
  .collect(Collectors.groupingBy(X::feld, Collectors.averagingInt(X::wert)))

sortieren und begrenzen:
  .sorted(Comparator.comparingInt(X::wert)).limit(3).toList()
  .max(Comparator.comparingInt(X::wert))
```

Zwei Sonderfälle: Für „je Gruppe das Maximum" brauchst du groupingBy mit collectingAndThen und maxBy. Für „sortiert nach Häufigkeit" gruppierst du erst und sortierst dann die entrySet nach dem Wert.

So oft wie möglich – das sind Einzeiler.

Durchgänge: ☐ ☐ ☐ ☐

Der C-Teil am Schluss

Laut Protokoll kam zum Schluss noch eine C-Aufgabe, die der Schreiber nicht mehr geschafft hat. Eschs Folie sagt: eine oder zwei einfache C-Aufgaben.

Aus SS 2019 und SS 2024 wissen wir, wie sie aussieht: eine rekursive Funktion, die ein Ergebnis über einen Pointer zurückgibt. Der komplette Block steht im Drillplan, Teil 3.

```
void findMax(int* arr, int n, int i, int* max) {
    if (i >= n) return;
    if (arr[i] > *max) *max = arr[i];
    findMax(arr, n, i + 1, max);
}
// int max = numbers[0];  findMax(numbers, count, 0, &max);
```

F1 aus dem Drillplan an drei Tagen je einmal.

Durchgänge: ☐ ☐ ☐

Teil 2 – Die wiederkehrenden MC-Fragen

Wie du damit übst

Erst ankreuzen, dann den grünen Kasten aufdecken. Keine Minuspunkte in der Klausur – also im Zweifel alles ankreuzen, was plausibel ist.

Die ersten fünf Fragen kamen in mehreren Klausuren wörtlich identisch vor. Die restlichen sind Standardfragen zu den Themen mit hoher Gewichtung.

Die fünf Wiederholungstäter

W1 Welche der folgenden Lambda-Ausdrücke sind syntaktisch korrekt?

- ☐ (a, b) -> a+b
- ☐ (a, b) -> return a+b
- ☐ (a, b) -> return a+b;
- ☐ (a, b) -> (return a+b;)
- ☐ a, b -> a+b
- ☐ int a, int b -> a+b
- ☐ a, b -> a+b;
- ☐ (int a, int b) -> a+b

Antwort

Richtig: (a, b) -> a+b und (int a, int b) -> a+b

Klammern um die Parameter darf man nur weglassen, wenn es genau ein Parameter ohne Typangabe ist. return gibt es nur zusammen mit geschweiften Klammern. Also wäre auch (a, b) -> { return a+b; } korrekt – das stand hier aber nicht zur Auswahl.

W2 class Container<T extends Number> mit Konstruktoren Container(T arg) und Container(Container<T> other). Welche Instanziierungen sind korrekt?

- ☐ Container<Integer> e1 = new Container<Integer>(new Integer(42));
- ☐ Container<Number> e2 = new Container<Number>(c1);
- ☐ Container<Number> e3 = new Container<Integer>(c1);
- ☐ Container<Integer> e4 = new Container<Integer>(c1);

Antwort

Richtig: e1 und e4

e1 ist eine normale Instanziierung mit einem Integer. e4 nutzt den Copy-Konstruktor mit einem Container<Integer> – die Typen passen. e3 scheitert, weil Generics invariant sind: Container<Integer> ist kein Container<Number>. e2 scheitert, weil c1 vom Typ Container<Integer> ist und nicht Container<Number>.

W3 long l = …; long* pl = &l; Ausgaben von &*pl, &pl und *pl sind 0x7ffee9ca2b38, 0x7ffee9ca2b30, 0x7ffee905eb3c. Welche ist die Adresse von l?

- ☐ 0x7ffee9ca2b38
- ☐ 0x7ffee9ca2b30
- ☐ 0x7ffee905eb3c

Antwort

Die erste: 0x7ffee9ca2b38

&*pl ist dasselbe wie pl, also die Adresse von l. Die zweite ist &pl, die Adresse des Zeigers selbst. Die dritte

ist *pl, also der Wert von l – der wird hier nur fälschlich mit %p ausgegeben.

W3b Welche Aussagen zu demselben Code treffen zu?

- ☐ &*pl und pl liefern die gleiche Adresse.
- ☐ &*pl liefert die Adresse von l.
- ☐ &pl liefert die Adresse des Zeigers selbst.
- ☐ *pl dereferenziert den Zeiger und gibt den Wert von l.
- ☐ Alle drei Ausgaben sind identisch.

Antwort

Die ersten vier sind richtig, die letzte ist falsch.

W4 a.findMax(new ArrayList<Integer>(), a::countDigits) und a.findMax(..., Integer::highestOneBit). Welche Form der Methodenreferenz liegt jeweils vor?

- ☐ Referenzen zu statischen Methoden
- ☐ Referenzen zu Instanz-Methoden eines gegebenen Typs
- ☐ Referenzen zu Instanz-Methoden eines Objekts
- ☐ Referenzen zu Konstruktoren

Antwort

a::countDigits → Instanzmethode eines Objekts. Integer::highestOneBit → statische Methode.

Entscheidend ist, was links vom :: steht. Ein Objekt (kleingeschriebene Variable) bedeutet Instanzmethode eines bestimmten Objekts. Ein Klassenname mit statischer Methode bedeutet statisch. Ein Klassenname mit Instanzmethode – etwa String::length – wäre die dritte Variante.

W5 Welche Verwendung von scanf zum Einlesen eines int i ist korrekt?

- ☐ scanf("%d", *i);
- ☐ scanf("%d", &i);
- ☐ scanf("%d", i);
- ☐ scanf(i);
- ☐ scanf("%i");

Antwort

Richtig: scanf("%d", &i);

scanf braucht die Adresse, um den Wert hineinschreiben zu können. Bei printf ist es umgekehrt: printf("%d", i) ohne &, weil dort nur der Wert gelesen wird.

Rekursion

R1 Wann kommt eine Rekursion zu Ende?

- ☐ Wenn der Basisfall erreicht ist.
- ☐ Wenn der Stack überläuft.
- ☐ Wenn der Rückgabewert null ist.
- ☐ Wenn die Bedingung der Rekursion false wird.
- ☐ Wenn keine weiteren Methoden mehr aufgerufen werden.

Antwort

Richtig: Basisfall erreicht und Bedingung wird false

Beides beschreibt dasselbe. Ein Stack-Überlauf beendet die Rekursion nicht ordentlich, sondern bricht das Programm ab.

R2 Wo werden die rekursiven Aufrufe gespeichert?

- ☐ Im Heap.
- ☐ Im Stack.
- ☐ In globalen Variablen.
- ☐ Im Datenbereich des Programms.
- ☐ In der PermGen bzw. im Metaspace.

Antwort

Im Stack.

Jeder Aufruf legt einen eigenen Stack-Frame mit eigenen lokalen Variablen und Parametern an. Bei unendlicher Rekursion läuft der Stack über: StackOverflowError.

R3 Kann eine rekursive Funktion auch iterativ berechnet werden?

- ☐ Ja, immer
- ☐ Nein, nie
- ☐ Manchmal

Antwort

Ja, immer.

Jede Rekursion lässt sich in eine Iteration umschreiben und umgekehrt. Manchmal braucht man dafür einen eigenen Stack, möglich ist es aber immer.

Innere Klassen

I1 Was sind Eigenschaften von inneren Klassen?

- ☐ Haben Zugriff auf private Felder der äußeren Klasse.
- ☐ Können static oder non-static sein.
- ☐ Müssen in einer eigenen Datei stehen.
- ☐ Können von Interfaces erben.
- ☐ Dürfen nicht public sein.

Antwort

Richtig: Zugriff auf private Felder und static oder non-static möglich

Dass sie NICHT in einer eigenen Datei stehen müssen, ist gerade ihr Zweck. Sie dürfen auch public sein.

I2 Worauf kann man aus einer inneren Klasse heraus zugreifen?

- ☐ Auf public Member der umgebenden Klasse
- ☐ Auf private Member der umgebenden Klasse
- ☐ Auf protected Member der umgebenden Klasse

Antwort

Auf alle drei.

Das ist der eigentliche Vorteil innerer Klassen: Sie kommen an alles heran, auch an private Felder.

I3 Auf welche Variablen kann aus einer lokalen Klasse zugegriffen werden?

- ☐ Auf lokale Variablen der Methode, wenn sie effektiv final sind.
- ☐ Auf Parameter der Methode, wenn sie effektiv final sind.
- ☐ Auf statische Variablen anderer Klassen.
- ☐ Auf globale Variablen.
- ☐ Auf lokale Variablen anderer Methoden.

Antwort

Richtig: die ersten drei.

Effektiv final heißt: nach der Initialisierung nicht mehr verändert. Globale Variablen gibt es in Java nicht, und an lokale Variablen anderer Methoden kommt niemand heran.

I4 Was sind Eigenschaften von anonymen Klassen?

- ☐ Haben keinen Klassennamen.
- ☐ Werden direkt bei der Objekterzeugung geschrieben.
- ☐ Können nur einmal instanziiert werden.
- ☐ Können Konstruktorparameter verwenden.
- ☐ Können mehrere Interfaces gleichzeitig implementieren.

Antwort

Richtig: die ersten vier.

Eine anonyme Klasse kann genau EIN Interface implementieren oder EINE Klasse erweitern – nicht mehrere gleichzeitig.

Generics

G1 Welche Eigenschaften haben Generics?

- ☐ Bieten Typprüfung zur Compilezeit.
- ☐ Unterstützen Typlöschung (Type Erasure).
- ☐ Können keine primitiven Typen wie int direkt verwenden.
- ☐ Reduzieren Casting zur Laufzeit.
- ☐ Können Wildcards als Platzhalter haben.

Antwort

Alle fünf sind richtig.

Type Erasure bedeutet: Nach der Typprüfung entfernt der Compiler die Typinformation. Daraus folgt alles Weitere – kein new T(), keine primitiven Typen, kein instanceof mit Typparameter.

G2 Welche Objekterzeugung ist erlaubt?

- ☐ GenericBox<Integer> gb = new GenericBox<Integer>();
- ☐ GenericBox<int> gb = new GenericBox<int>();
- ☐ GenericBox gb = new GenericBox();
- ☐ GenericBox<String> gb = new GenericBox<>();
- ☐ GenericBox<?> gb = new GenericBox<Object>();

Antwort

Alle außer der zweiten.

GenericBox<int> ist verboten – nur Klassentypen, also Integer. Die dritte ist ein Raw Type: erlaubt, aber der Compiler warnt.

G3 Was sind RAW Types?

- ☐ Generics ohne Typangabe.
- ☐ Veraltete Schreibweise von Generics.
- ☐ Typen, die keine Wildcards erlauben.
- ☐ Klassen, die nicht von Object erben.
- ☐ Datentypen für rohe Byte-Arrays.

Antwort

Generics ohne Typangabe.

Also List l = new ArrayList(); statt List<String>. Erlaubt aus Kompatibilitätsgründen zu Java vor Version 5, aber unsicher.

G4 Eigenschaften von Bounded Type Parametern?

- ☐ Können mit extends eingeschränkt werden.
- ☐ Können mit super eingeschränkt werden.
- ☐ Können nur Referenztypen verwenden.
- ☐ Dürfen keine Generics verschachteln.
- ☐ Erlauben Polymorphie innerhalb der Grenzen.

Antwort

Richtig: extends, super, nur Referenztypen, Polymorphie innerhalb der Grenzen.

Verschachteln ist erlaubt, etwa List<Map<String, Integer>>. new T() bleibt dagegen verboten.

Lambda und funktionale Interfaces

L1 Welches funktionale Interface nimmt ein Argument und gibt kein Ergebnis zurück?

- ☐ Consumer<T>
- ☐ Supplier<T>
- ☐ Function<T,R>
- ☐ Predicate<T>
- ☐ BiFunction<T,U,R>

Antwort

Consumer<T>

Die Methode heißt void accept(T).

L2 Welches funktionale Interface nimmt keine Argumente und gibt ein Ergebnis zurück?

- ☐ Consumer<T>
- ☐ Supplier<T>
- ☐ Function<T,R>
- ☐ Predicate<T>

Antwort

Supplier<T>

Die Methode heißt T get().

L3 Welches funktionale Interface nimmt zwei Argumente vom Typ T und gibt T zurück?

- ☐ BinaryOperator<T>
- ☐ BiConsumer<T,T>
- ☐ BiFunction<T,T,T>
- ☐ Supplier<T>
- ☐ Predicate<T>

Antwort

BinaryOperator<T>, formal auch BiFunction<T,T,T>

BinaryOperator ist die spezialisierte Kurzform, wenn alle drei Typen gleich sind.

Collections

C1 Welche Aussagen treffen auf List zu?

- ☐ Ist ein Interface.
- ☐ Enthält nur eindeutige Elemente.
- ☐ Erlaubt Duplikate.
- ☐ Hat eine feste Länge.
- ☐ Erhält Elemente in Einfügereihenfolge.

Antwort

Richtig: Interface, erlaubt Duplikate, Einfügereihenfolge.

Eine feste Länge hat nur ein Array, keine List.

C2 Welche Aussagen treffen auf ArrayList zu?

- ☐ ArrayList extends Collection
- ☐ ArrayList implements List
- ☐ ArrayList extends AbstractList
- ☐ ArrayList implements Comparator
- ☐ ArrayList ist threadsicher

Antwort

Richtig: implements List und extends AbstractList

Collection ist ein Interface, das wird implementiert und nicht erweitert. Comparator hat mit ArrayList nichts zu tun, und threadsicher ist sie nicht.

C3 Welche Aussagen treffen auf TreeMap zu?

- ☐ Die Schlüssel sind sortiert.
- ☐ Die Schlüssel müssen Comparable sein oder ein Comparator muss übergeben werden.
- ☐ TreeMap ist ein Untertyp von Collection.
- ☐ TreeMap erlaubt doppelte Schlüssel.

Antwort

Richtig: die ersten beiden.

Map ist KEIN Untertyp von Collection – beliebte Fangfrage. Und Schlüssel sind in jeder Map eindeutig.

Streams

S1 Welche Terminal-Operationen gibt es?

- ☐ forEach()
- ☐ map()
- ☐ reduce()
- ☐ collect()
- ☐ filter()

Antwort

Richtig: forEach, reduce, collect.

map und filter geben wieder einen Stream zurück, sind also intermediate. Merkregel: Was einen Stream zurückgibt, ist intermediate. Was einen Wert, eine Collection oder nichts zurückgibt, ist terminal.

S2 Welche Ausgabe erzeugt Stream.of(1, 1, 3, 4).distinct().sorted().toList()?

- ☐ 3
- ☐ 6
- ☐ 1, 1, 3, 4
- ☐ 1, 3, 4
- ☐ Compilerfehler

Antwort

[1, 3, 4]

distinct entfernt das Duplikat, sorted sortiert. Die 3 wäre die Anzahl, nicht die Liste.

C

K1 Welche Eigenschaften besitzt C?

- ☐ hohe Typsicherheit
- ☐ manuelle Speicherverwaltung
- ☐ Plattformunabhängigkeit
- ☐ prozedurale Programmierung
- ☐ funktionale Programmierung
- ☐ kompiliert direkt zu Maschinencode
- ☐ unterstützt Zeigerarithmetik
- ☐ hat Garbage Collection
- ☐ unterstützt Klassen und Objekte

Antwort

Richtig: manuelle Speicherverwaltung, prozedural, Maschinencode, Zeigerarithmetik.

C ist nicht typsicher (kein Bounds Checking), nicht plattformunabhängig (kompiliert für eine Zielplattform), hat keine Garbage Collection und keine Klassen.

K2 Was ist die Aufgabe eines Linkers?

- ☐ Verknüpft Objektdateien zu einem Programm.
- ☐ Übersetzt Quellcode zu Bytecode.
- ☐ Lädt das Programm in den Speicher.
- ☐ Optimiert den Code zur Compilezeit.

☐ Verknüpft Bibliotheken zur Laufzeit.

Antwort

Verknüpft Objektdateien zu einem Programm.

Bytecode erzeugt javac, nicht der Linker. Ins Speicher laden macht der Loader des Betriebssystems.

K3 Mit welchem Wort definiert man eine Structure in C?

- ☐ class
- ☐ typedef
- ☐ struct
- ☐ enum
- ☐ record

Antwort

struct.

typedef vergibt nur zusätzlich einen kurzen Typnamen – das Schlüsselwort für die Struktur selbst ist struct.

K4 Mit welchem Operator fragt man in C die Größe einer Variablen ab?

- ☐ #
- ☐ &
- ☐ *
- ☐ getsize
- ☐ sizeof
- ☐ varsize

Antwort

sizeof

K5 Welche der folgenden Datentypen gibt es in C?

- ☐ string
- ☐ float
- ☐ byte
- ☐ short int
- ☐ int *
- ☐ boolean
- ☐ long long

Antwort

Richtig: float, short int, int *, long long.

string gibt es nicht – Strings sind char-Arrays. byte und boolean gibt es auch nicht; ohne stdbool.h arbeitet man mit int, wobei 0 falsch ist und alles andere wahr.

Teil 3 – Die Tabellen zum Nachschlagen

Die sechs Collections

Klasse	Duplikate	Reihenfolge	Wann nehmen
ArrayList	ja	Einfügereihenfolge	Standardliste
LinkedList	ja	Einfügereihenfolge	viel Einfügen und Löschen
HashSet	nein	keine	nur Eindeutigkeit gefragt
LinkedHashSet	nein	Einfügereihenfolge	eindeutig UND Reihenfolge
TreeSet	nein	sortiert	eindeutig UND sortiert
HashMap	Keys eindeutig	keine	schnelles Suchen über Schlüssel
LinkedHashMap	Keys eindeutig	Einfügereihenfolge	Map mit Reihenfolge
TreeMap	Keys eindeutig	nach Key sortiert	sortierte Map
ArrayDeque	ja	FIFO oder LIFO	Warteschlange oder Stack
PriorityQueue	ja	kleinstes zuerst	immer das Minimum entnehmen

Die drei Fragen, mit denen du jede Auswahl entscheidest

- Duplikate erlaubt? Nein → Set. Ja → List.
- Zugriff über einen Schlüssel? Ja → Map.
- Sortiert oder Einfügereihenfolge? Sortiert → Tree. Einfügereihenfolge → Linked. Egal → Hash.

Beispiel: „keine Duplikate, sortiert nach Rückgabedatum" → TreeSet mit Comparator. „unter einer ID gespeichert, in Einfügereihenfolge abrufbar" → LinkedHashMap.

Verifiziert mit derselben Eingabe (3, 1, 3)

```
ArrayList      [3, 1, 3]
HashSet        [1, 3]
TreeSet        [1, 3]
LinkedHashSet  [3, 1]
HashMap        {a=1, b=2}     Reihenfolge nicht garantiert
LinkedHashMap  {b=2, a=1}     wie eingefügt
TreeMap        {a=1, b=2}     nach Schlüssel sortiert
ArrayDeque     poll -> 5      FIFO, das zuerst Eingefügte
PriorityQueue  poll -> 1      das kleinste Element
```

Die Stream-Operationen

Intermediate	Terminal
filter, map, mapToInt, flatMap	forEach, collect, reduce, count, toList
sorted, distinct, limit, skip	min, max, sum, average, summaryStatistics
peek	anyMatch, allMatch, noneMatch, findFirst

Die Ketten, die du brauchst

```
liste.stream().filter(p).toList()
liste.stream().filter(p).count()
liste.stream().filter(p).sorted(c).toList()
liste.stream().map(X::getName).toList()
liste.stream().map(X::getName).collect(Collectors.joining(", "))
liste.stream().mapToInt(X::getWert).sum()
liste.stream().mapToInt(X::getWert).average().orElse(0)
liste.stream().collect(Collectors.groupingBy(X::getArt))
```

```
liste.stream().collect(Collectors.groupingBy(X::getArt, Collectors.counting()))
liste.stream().sorted(c).limit(3).toList()
liste.stream().max(Comparator.comparingInt(X::getWert))
liste.forEach(x -> System.out.println(x));

map.values().stream()....       // bei einer Map
parallelStream()                // wenn "parallel" verlangt wird
```

Die funktionalen Interfaces

Interface	Methode	Nimmt	Gibt zurück
`Supplier<T>`	`T get()`	nichts	T
`Consumer<T>`	`void accept(T)`	T	nichts
`Function<T,R>`	`R apply(T)`	T	R
`Predicate<T>`	`boolean test(T)`	T	boolean
`UnaryOperator<T>`	`T apply(T)`	T	T
`BinaryOperator<T>`	`T apply(T,T)`	zweimal T	T
`BiFunction<T,U,R>`	`R apply(T,U)`	T und U	R

Comparator und Predicate schreiben

```
Comparator.comparing(Car::getBrand)
Comparator.comparingInt(Car::getHorsePower)
Comparator.comparingInt(Car::getHorsePower).reversed()
Comparator.comparing(X::getA).thenComparing(X::getB)
Comparator.naturalOrder()

c -> c.getColor().equals("yellow")
c -> c.getColor().equals("yellow") && c.getBrand().equals("Lamborghini")
c -> c.getHorsePower() > 500
```

Code-Ausgaben, die geprüft wurden

Alle verifiziert

```
Stream.of(1,2,3).reduce(1, (x,y) -> x*y)          ->  6
Stream.of(1,2,3).reduce(0, (x,y) -> x+y)          ->  6
Stream.of(1,1,3,4).distinct().sorted().toList()   ->  [1, 3, 4]
List.of(1,2,3).stream().anyMatch(x -> x > 2)      ->  true

Function f = x -> x+1;  g = x -> x*10;
f.andThen(g).apply(3)                             ->  40
f.compose(g).apply(3)                             ->  31

fac(3) mit println("B") im Basisfall              ->  B     dann  6

int a[5] = {10,20,30,40,50};  int* p = a;
*(p+2)     ->  30        p[2]      ->  30
*(p)+2     ->  12        p++; *p   ->  20
```